

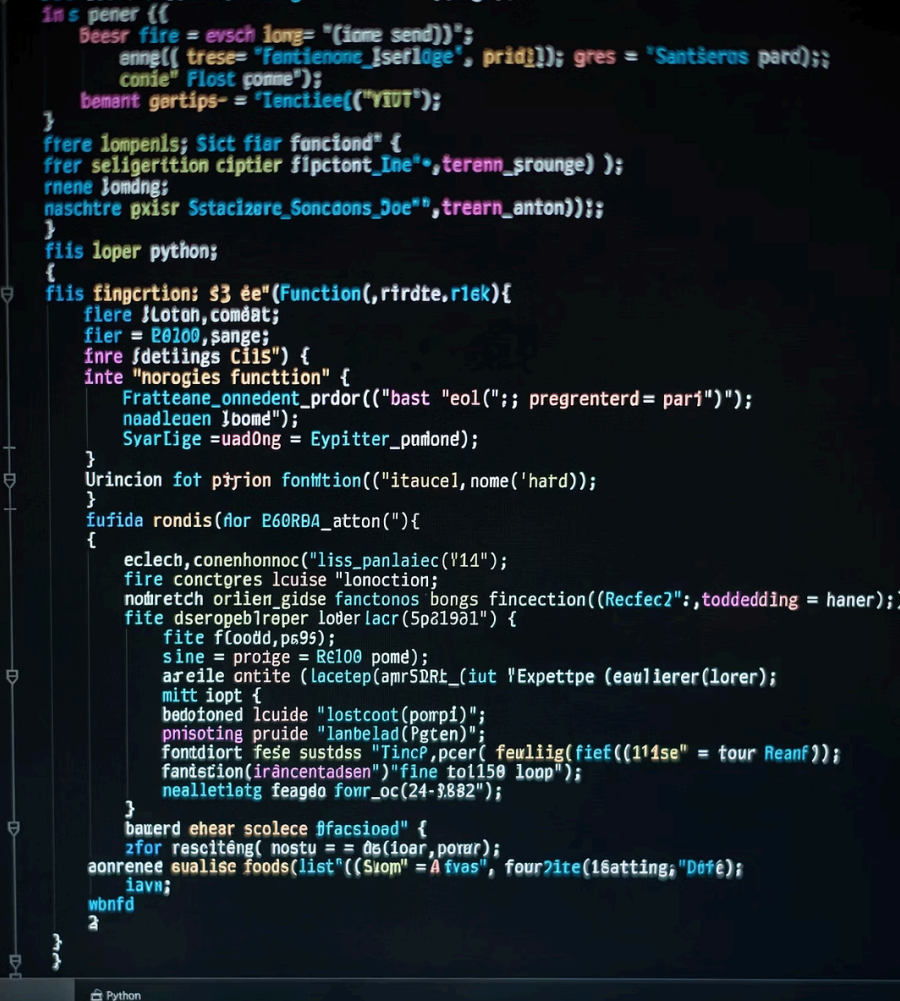
Python Fundamentals: Your First Steps into Programming

A beginner-friendly journey through the core concepts of one of the world's most popular and powerful programming languages — from variables to data structures and beyond.

BEGINNER FRIENDLY

HANDS-ON LEARNING





What is Python?

Python is an **interpreted, high-level programming language** celebrated for its clean, readable syntax and dynamic typing. Designed to emphasize code clarity, Python lets developers express complex ideas in fewer lines than many other languages — making it a top choice for beginners and experts alike.



Cross-Platform

Runs seamlessly on Windows, macOS, and Linux with no modification needed.



Rapid Development

Ideal for scripting, automation, and rapid application prototyping across domains.



Free & Open Source

Completely free to use and distribute, backed by a vast ecosystem of libraries and an active global community.

Variables: Giving Names to Data

Think of variables as **labeled containers** that store values in your program's memory. In Python, you assign a value using the `=` operator. Python is dynamically typed — meaning it *automatically infers* the data type without you having to declare it explicitly.

How Assignment Works

The variable name goes on the left, the value on the right. Python reads, evaluates, and stores it instantly.

- `age = 30` — stores the integer 30
- `name = "Alice"` — stores a string
- `price = 9.99` — stores a decimal number
- `is_active = True` — stores a boolean

Naming Rules

- Start with a letter or underscore, not a digit
- Use lowercase with underscores for clarity (`my_variable`)
- Avoid Python reserved keywords like `if`, `for`, `class`
- Names are case-sensitive: `Name` $\neq$ `name`

Data Types: The Building Blocks of Information

Every value in Python has a **data type** that defines what kind of information it holds and what operations can be performed on it. Python divides types into two critical categories: **immutable** (fixed after creation) and **mutable** (changeable after creation).

IMMUTABLE TYPES

10 **INT**

3.14 **FLOAT**

Hello **STR**

(1,2,3) **TUPLE**

MUTABLE TYPES

[1,2,3] **LIST**

{name: Bob} **DICT**

{1,2,3} **SET**

❗ A key rule: **immutable objects** cannot be altered in place — any "change" creates a new object. **Mutable objects** can be modified directly, which makes them flexible but requires careful handling to avoid unintended side effects.

Control Flow: Making Decisions in Your Code

Control flow structures are the **decision-making engine** of any program. Python provides `if/elif/else` for branching logic, `for` loops for iterating over sequences, and `while` loops for repeating until a condition changes.

1

`if / elif / else`

Evaluates conditions in order. Executes the first block whose condition is `True`. The `else` block is a catch-all fallback when no conditions match.

2

`for` Loop

Iterates over any sequence — lists, strings, ranges, or dictionaries. Runs the indented block once per item: `for item in sequence:`

3

`while` Loop

Continues executing as long as its condition remains `True`. Always ensure the condition will eventually become `False` to avoid infinite loops.

```
score = 85
if score >= 90:
    print("A")
elif score >= 75:
    print("B") # This runs!
else:
    print("C")
```




Lists: Your Versatile Ordered Collection

A **list** is Python's most flexible built-in data structure — an *ordered, mutable* sequence that can hold items of any type, even mixed types in the same list. Defined with square brackets `[]`, lists are the go-to tool whenever you need a changeable, ordered group of values.

Example

```
my_list = [10, "apple", 3.14, True]
```

Lists maintain insertion order and support indexing starting at 0. Access any element with `my_list[0]` → returns 10.

Why Lists?

- Store multiple related values under one name
- Grow or shrink dynamically at runtime
- Support slicing, sorting, and iteration natively
- Nest lists within lists for 2D structures

Key List Methods: Manipulating Your Data

Python lists come loaded with built-in methods that make adding, removing, and organizing data intuitive. These methods modify the list **in place** — no new list is created — keeping your code efficient and readable.



append() & insert()

`append(item)` adds to the end.
`insert(index, item)` places an item at any specific position without overwriting existing elements.



remove() & pop()

`remove(item)` deletes the first occurrence by value. `pop()` removes and *returns* the last item — or any index you specify.



sort() & len()

`sort()` rearranges items in ascending order in place. `len(list)` returns the total item count — essential for loops and validation.

```
fruits = ["banana", "apple", "cherry"]
fruits.append("mango") # ['banana','apple','cherry','mango']
fruits.sort() # ['apple','banana','cherry','mango']
fruits.pop() # removes 'mango', returns it
print(len(fruits)) # 3
```

Tuples: Immutable Ordered Sequences

Tuples look like lists but with one fundamental difference: they are **immutable**. Once a tuple is created with parentheses `()`, its contents are locked — no additions, removals, or changes allowed. This makes tuples the ideal choice for data that must remain constant throughout a program's life.

Fixed Coordinates

```
location = (40.7128, -74.0060)
```

GPS points should never accidentally change — tuples enforce this guarantee.

RGB Color Values

```
blue = (0, 0, 255)
```

Color constants stay consistent across a design system when stored as tuples.

Config Settings

```
config = ("localhost", 8080)
```

Server settings and fixed configurations are safer as immutable sequences.

 Tuples are also slightly **faster** than lists and can be used as **dictionary keys** — a capability lists do not have, since keys must be immutable.

Dictionaries: Key-Value Pair Powerhouses

Dictionaries (dict) store data as **key-value pairs** enclosed in curly braces {}. They allow you to look up values instantly by a meaningful key rather than a numeric index — making them one of the most powerful and widely-used data structures in Python.

Rules to Remember

- Keys must be **unique** — duplicates overwrite the earlier value
- Keys must be **immutable** (strings, numbers, or tuples)
- Values can be *any* data type, including lists or other dicts
- Access with `dict['key']` or `dict.get('key')`

Practical Example

```
student = {  
    'name': 'Charlie',  
    'major': 'Computer Science',  
    'gpa': 3.8,  
    'courses': ['Python', 'ML']  
}  
  
print(student['name']) # Charlie  
student['gpa'] = 3.9 # update value  
student['year'] = 2 # add new key
```

Your Python Journey Begins!

You've built a solid foundation in Python's core concepts. These fundamentals are the bedrock upon which every Python application — from web servers to machine learning models — is built. Keep practicing and the path forward becomes clearer with every line of code.

01

Variables & Types

Understand how Python stores data using named variables and recognizes types like `int`, `str`, and `float` automatically.

02

Mutable vs. Immutable

Know the difference between data that can change (lists, dicts, sets) and data that stays fixed (`int`, `str`, `tuple`).

03

Control Flow

Use `if/elif/else`, `for`, and `while` to guide your program's logic and decisions dynamically.

04

Collections

Leverage lists, tuples, and dictionaries with their powerful built-in methods to organize and manipulate data efficiently.

✅ **Next Steps:** Practice by building small projects — a to-do list app, a number guessing game, or a simple contact book. Real coding is the fastest path to mastery! 🚀